

Practical 1

Aim: To perform data analysis using Conditional Formatting, Pivot Tables, VLOOKUP, and Goal Seek.

1. Conditional Formatting

1. Select the column containing numerical data (e.g., **Percentage**).
2. Navigate to **Home > Conditional Formatting > Highlight Cells Rules > Greater Than**.
3. Enter a threshold value (e.g., **80**) and choose a formatting style (e.g., **Green Fill**).
4. Click **OK** to highlight cells meeting the criteria.

2. Pivot Table

1. Select the entire dataset range.
2. Go to **Insert > PivotTable** and choose **New Worksheet**.
3. In the **PivotTable Fields** pane, drag a categorical field (e.g., **Name**) to the **Rows** area.
4. Drag a numerical field (e.g., **Total Marks**) to the **Values** area to summarize the data.

3. VLOOKUP Function

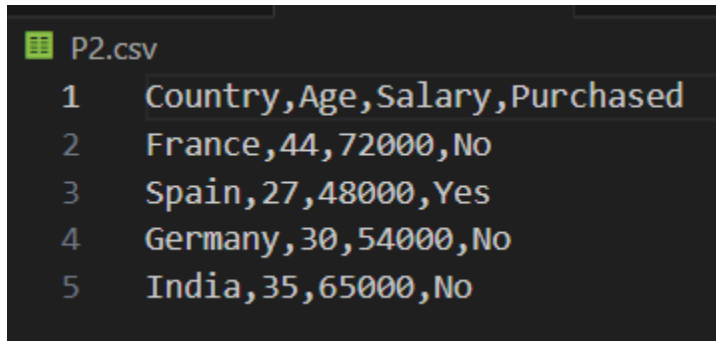
1. Designate a cell for the lookup value (e.g., **K2**) and a cell for the result.
2. In the result cell, enter the formula: `=VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])`
3. *Example:* `=VLOOKUP(K2, A2:I8, 9, FALSE)`
 - **K2**: The name to search for.
 - **A2:I8**: The full data table range.
 - **9**: The column number containing the return value (Percentage).
 - **FALSE**: Ensures an exact match.

4. Goal Seek Analysis

1. Select the cell containing the calculated formula (e.g., **Percentage**).
2. Navigate to **Data > What-If Analysis > Goal Seek**.
3. In the dialog box, configure the following:
 - **Set cell**: The cell with the formula (e.g., Percentage).
 - **To value**: The desired target value (e.g., **97**).
 - **By changing cell**: The input cell to adjust (e.g., **Subject Marks**).
4. Click **OK** to calculate the required value.

Practical 2

- Aim: Read data from CSV and JSON files into a data frame and perform basic data pre-processing tasks such as handling missing values and outliers.



	Country	Age	Salary	Purchased
1	France	44	72000	No
2	Spain	27	48000	Yes
3	Germany	30	54000	No
4	India	35	65000	No

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

ds = pd.read_csv("P2.csv")
print("Data Head:\n",ds.head())
print("Data Describe:\n",ds.describe())

X = ds.iloc[:, :-1].values
Y = ds.iloc[:, 3].values
print("\nInput:",X)
print("\nOutput:",Y)

# HANDLING MISSING VALUES
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(missing_values = np.nan, strategy =
"mean")
imputer = imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.fit_transform(X[:, 1:3])
print("\nNew Input with mean value for Nan:",X)

# OUTLIERS
import sklearn
from sklearn.datasets import load_diabetes
import pandas as pd
import matplotlib.pyplot as plt

db = load_diabetes()

column_name = db.feature_names
df_db = pd.DataFrame(db.data)
df_db.columns = column_name
df_db.head()
```

```

import seaborn as sns

sns.boxplot(df_db['bmi'])

import numpy as np

print(np.where(df_db['bmi']>0.12))

# SORTING
print(df_db)
sorted = df_db.sort_values(by = ["age"])
print(sorted)

# FILTERING ROWS
a = df_db.query('age>0')
print(a)
# FILTERING COLUMNS
b = df_db.filter(['age', 'bp'])
print(b)
# GROUPING DATA
g = df_db.groupby('age')
g.first()

print("DONE")

```

	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	India	35	65000	No

Data Describe:

	Age	Salary
count	4.000000	4.000000
mean	34.000000	59750.000000
std	7.438638	10781.929326
min	27.000000	48000.000000
25%	29.250000	52500.000000
50%	32.500000	59500.000000
75%	37.250000	66750.000000
max	44.000000	72000.000000

Input: [['France' 44 72000]
['Spain' 27 48000]
['Germany' 30 54000]
['India' 35 65000]]

Output: ['No' 'Yes' 'No' 'No']

[442 rows x 10 columns]

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
374	-0.107226	-0.044642	-0.034229	-0.067642	-0.063487	-0.070520	0.008142	-0.039493	-0.000612	-0.079778
344	-0.107226	-0.044642	-0.011595	-0.040099	0.049341	0.064447	-0.013948	0.034309	0.007027	-0.030072
26	-0.107226	-0.044642	-0.077342	-0.026328	-0.089630	-0.096198	0.026550	-0.076395	-0.042571	-0.005220
79	-0.103593	-0.044642	-0.037463	-0.026328	0.002559	0.019980	0.011824	-0.002592	-0.068332	-0.025930
226	-0.103593	0.050680	-0.046085	-0.026328	-0.024960	-0.024800	0.030232	-0.039493	-0.039809	-0.054925
..	...	...	...	...	...	...	...	...	...	...
211	0.092564	-0.044642	0.036907	0.021872	-0.024960	-0.016658	0.000779	-0.039493	-0.022517	-0.021788
311	0.096197	-0.044642	0.040140	-0.057313	0.045213	0.060690	-0.021311	0.036154	0.012551	0.023775
321	0.096197	-0.044642	0.051996	0.079265	0.054845	0.036577	-0.076536	0.141322	0.098648	0.061054
402	0.110727	0.050680	-0.033151	-0.022885	-0.004321	0.020293	-0.061809	0.071210	0.015568	0.044485
204	0.110727	0.050680	0.006728	0.028758	-0.027712	-0.007264	-0.047082	0.034309	0.002004	0.077622

[442 rows x 10 columns]

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
7	0.063504	0.050680	-0.001895	0.066629	0.090620	0.108914	0.022869	0.017703	-0.035816	0.003064
8	0.041708	0.050680	0.061696	-0.040099	-0.013953	0.006202	-0.028674	-0.002592	-0.014960	0.011349
..	...	...	...	...	...	...	...	...	...	...
431	0.070769	0.050680	-0.030996	0.021872	-0.037344	-0.047034	0.033914	-0.039493	-0.014960	-0.001078
432	0.009016	-0.044642	0.055229	-0.005670	0.057597	0.044719	-0.002903	0.023239	0.055686	0.106617
434	0.016281	-0.044642	0.001339	0.008101	0.005311	0.010899	0.030232	-0.039493	-0.045424	0.032059
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491

[240 rows x 10 columns]

	age	bp
0	0.038076	0.021872
1	-0.001882	-0.026328
2	0.085299	-0.005670
3	-0.089063	-0.036656
4	0.005383	0.021872
..	...	...
437	0.041708	0.059744
438	-0.005515	-0.067642
439	0.041708	0.017293
440	-0.045472	0.001215
441	-0.045472	-0.081413

[442 rows x 2 columns]

DONE

Practical 3

Aim: Manipulate and transform data using functions like filtering, sorting, and grouping. Perform feature dummification to convert categorical variables into numerical representations and apply feature-scaling techniques like standardization and normalization

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler, MinMaxScaler
import matplotlib.pyplot as plt

iris = load_iris()
data =
pd.DataFrame(data=np.c_[iris['data'],iris['target']],columns=iris['feature_names']+[ 'target'])

numerical_features = iris['feature_names']
X = data[numerical_features]
Y = data['target']

scaler_standard = StandardScaler()
X_standardized = scaler_standard.fit_transform(X)

scaler_minmax = MinMaxScaler()
X_normalized = scaler_minmax.fit_transform(X)

plt.figure(figsize=(12,4))

plt.subplot(131)
plt.scatter(X.iloc[:,0],X.iloc[:,1],c=Y,cmap='viridis')
plt.title('Original Features')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.subplot(132)
plt.scatter(X_standardized[:,0],X_standardized[:,1],c=Y,cmap='viridis')
plt.title('Standardized Features')
plt.xlabel('Feature 1(Standardized)')
plt.ylabel('Feature 2(Standardized)')

plt.subplot(133)
plt.scatter(X_normalized[:,0],X_normalized[:,1],c=Y,cmap='viridis')
plt.title('Standardized Features')
plt.xlabel('Feature 1(Normalized)')
plt.ylabel('Feature 2(Normalized)')

plt.tight_layout()
plt.show()
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = {'Color':['Red','Blue','Green','Red','Blue'],
        'Size':['Small','Large','Medium','Medium','Small'],
```

```

        'Label':[1,0,1,0,1]
    }

df = pd.DataFrame(data)

df_encoded = pd.get_dummies(df,columns=['Color','Size'],drop_first=True)

print("Original DataFrame:")
print(df)
print("\nDataFrame after feature dummification:")
print(df_encoded)

X = df_encoded.drop('Label',axis = 1)
Y = df_encoded['Label']

X_train,X_test,Y_train,Y_test =
train_test_split(X,Y,test_size=0.2,random_state=42)

model = RandomForestClassifier(random_state=42)
model.fit(X_train,Y_train)

Y_pred = model.predict(X_test)

accuracy = accuracy_score(Y_test,Y_pred)
print("\nModel Accuracy:",accuracy)

```

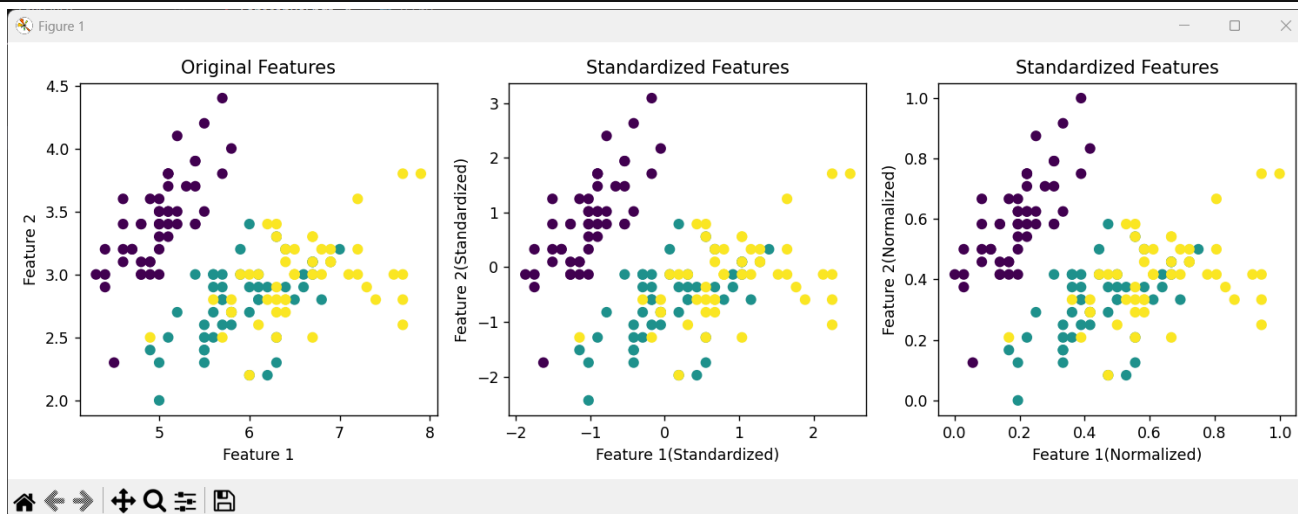
```

(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> & C:/Users/tembu/OneDrive/Desktop/DataScience/venv/Scripts/python.exe c:/Users/tembu/OneDrive/Desktop/DataScience/P3.py
Original DataFrame:
   Color  Size  Label
0    Red  Small     1
1   Blue  Large     0
2  Green  Medium     1
3    Red  Medium     0
4   Blue  Small     1

DataFrame after feature dummification:
   Label  Color_Green  Color_Red  Size_Medium  Size_Small
0      1           False         True         False         True
1      0           False         False         False         False
2      1           True         False         True         False
3      0           False         True         True         False
4      1           False         False         False         True

Model Accuracy: 0.0
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience>

```



Practical 4

Aim: Formulate null and alternative hypotheses for a given problem, conduct a hypothesis test using appropriate statistical tests (e.g., t-test, chi-square test), and interpret the results to draw conclusions

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

np.random.seed(42)
sample1 = np.random.normal(loc=10, scale=2, size=30)
sample2 = np.random.normal(loc=12, scale=2, size=30)

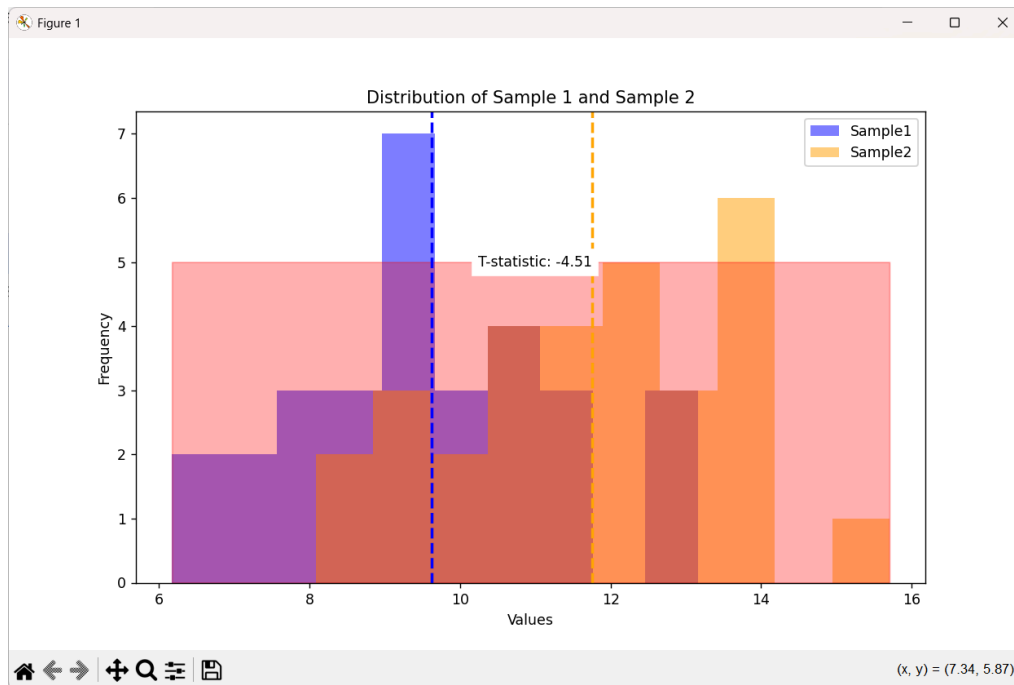
t_statistic, p_value = stats.ttest_ind(sample1, sample2)

alpha = 0.05
print("results of Two-Sample T-test")
print(f"t-statistic: {t_statistic}")
print(f"p-value: {p_value}")
print(f"Degrees of Freedom: {len(sample1)+len(sample2)-2}")

plt.figure(figsize=(10,6))
plt.hist(sample1, alpha=0.5, label="Sample1", color='blue')
plt.hist(sample2, alpha=0.5, label="Sample2", color='orange')
plt.axvline(np.mean(sample1), color='blue', linestyle='dashed', linewidth=2)
plt.axvline(np.mean(sample2), color='orange', linestyle='dashed', linewidth=2)
plt.title("Distribution of Sample 1 and Sample 2")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.legend()

if p_value < alpha:
    critical_region =
np.linspace(min(sample1.min(), sample2.min()), max(sample1.max(), sample2.max()), 1000)
    plt.fill_between(critical_region, 0, 5, color='red', alpha=0.3, label="Critical Region")
    plt.text(11, 5, f"T-statistic: {t_statistic:.2f}", ha="center", va="center", color="black", backgroundcolor="white")
    plt.show()

    if p_value < alpha:
        if np.mean(sample1) > np.mean(sample2):
            print("Conclusion: There is significant evidence to reject the null hypothesis")
            print("Interpretation: The mean caffeine content of Sample1 is significantly higher than that of Sample2")
        else:
            print("Conclusion: There is significant evidence to reject the null hypothesis")
            print("Interpretation: The mean caffeine content of Sample2 is significantly higher than that of Sample1")
        else:
            print("Conclusion: Fail to reject the null hypothesis")
            print("Interpretation: There is not enough evidence to claim a significance difference between the means")
```



```
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> & C:/Users/tembu/OneDrive/Desktop/DataScience
results of Two-Sample T-test
t-statistic: -4.512913234547555
p-value: 3.176506547470154e-05
Degrees of Freedom: 58
Conclusion: There is significant evidence to reject the null hypothesis
Interpretation: The mean caffeine content of Sample2 is significantly higher than that of Sample1
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience>
```

Practical 5

Aim: Perform one-way ANOVA to compare means across multiple groups and conduct post-hoc tests to identify significant differences between group means.

```
import scipy.stats as stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd

group1 = [23,25,29,34,30]
group2 = [19,20,22,25,24]
group3 = [15,18,20,21,17]
group4 = [28,24,26,30,29]

all_data = group1+group2+group3+group4

group_labels =
['Group1']*len(group1)+['Group2']*len(group2)+['Group3']*len(grou
p3)+['Group4']*len(group4)

f_statistic,p_value = stats.f_oneway(group1,group2,group3,group4)

print("One-way ANOVA: ")
print("F-statistics:",f_statistic)
print("P-value:"),p_value

tukey_results = pairwise_tukeyhsd(all_data,group_labels)

print("\nTukey-Kramer post-hoc test:")
print(tukey_results)
```

● One-way ANOVA:

F-statistics: 12.139872842870115

P-value: 0.00021465200901629603

Tukey-Kramer post-hoc test:

Multiple Comparison of Means - Tukey HSD, FWER=0.05

```
=====
group1 group2 meandiff p-adj  lower  upper  reject
-----
Group1 Group2      -6.2  0.024 -11.6809 -0.7191  True
Group1 Group3     -10.0 0.0004 -15.4809 -4.5191  True
Group1 Group4      -0.8 0.9747  -6.2809  4.6809  False
Group2 Group3      -3.8 0.2348  -9.2809  1.6809  False
Group2 Group4       5.4 0.0542  -0.0809 10.8809  False
Group3 Group4       9.2 0.001   3.7191 14.6809  True
-----
```

○ (venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> █

Practical 6

Aim: Implement simple linear regression, explore and interpret regression model coefficients and goodness-of-fit measures, and extend the analysis to multiple linear regression to assess the impact of additional predictors.

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

housing = fetch_california_housing()
housing_df = pd.DataFrame(housing.data, columns=housing.feature_names)
print(housing_df)
housing_df['PRICE'] = housing.target

X = housing_df[['AveRooms']]
Y = housing_df['PRICE']
X_train, X_test, Y_train, Y_test =
train_test_split(X, Y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, Y_train)

Y_pred = model.predict(X_test)

mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)
print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_)

X = housing_df.drop('PRICE', axis=1)
Y = housing_df['PRICE']
X_train, X_test, Y_train, Y_test =
train_test_split(X, Y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

print("MSE:", mse)
print("R-squared:", r2)
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• (venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> & C:/Users/tembu/OneDrive/Desktop/DataScience/Scripts/fit_model.py
MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude Longitude
0      8.3252      41.0  6.984127  1.023810      322.0  2.555556      37.88      -122.23
1      8.3014      21.0  6.238137  0.971880     2401.0  2.109842      37.86      -122.22
2      7.2574      52.0  8.288136  1.073446      496.0  2.802260      37.85      -122.24
3      5.6431      52.0  5.817352  1.073059      558.0  2.547945      37.85      -122.25
4      3.8462      52.0  6.281853  1.081081      565.0  2.181467      37.85      -122.25
...      ...      ...      ...      ...      ...      ...      ...
20635  1.5603      25.0  5.045455  1.133333      845.0  2.560606      39.48      -121.09
20636  2.5568      18.0  6.114035  1.315789      356.0  3.122807      39.49      -121.21
20637  1.7000      17.0  5.205543  1.120092     1007.0  2.325635      39.43      -121.22
20638  1.8672      18.0  5.329513  1.171920      741.0  2.123209      39.43      -121.32
20639  2.3886      16.0  5.254717  1.162264     1387.0  2.616981      39.37      -121.24

[20640 rows x 8 columns]
Mean Squared Error: 1.2923314440807299
R-squared: 0.013795337532284901
Intercept: 1.654762268596842
Coefficient: [0.07675559]
MSE: 0.5558915986952435
R-squared: 0.5757877060324514
Intercept: -37.023277706064256
Coefficients: [ 4.48674910e-01  9.72425752e-03 -1.23323343e-01  7.83144907e-01
 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]
○ (venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience>
```

Practical 7

Aim: Build a logistic regression model to predict a binary outcome, evaluate the model's performance using classification metrics (accuracy, precision, recall), and construct a decision tree model to interpret decision rules.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
classification_report

# Load the Iris dataset
iris = load_iris()

# Create the dataframe
iris_df = pd.DataFrame(data= np.c_[iris['data'], iris['target']], columns=
iris['feature_names'] + ['target'])

# For binary classification, let's consider only two classes (0 and 1)
binary_df = iris_df[iris_df['target'] != 2]

# Selecting features and target variable
X = binary_df.drop('target', axis=1)
y = binary_df['target']

# Splitting the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Logistic Regression model
logistic_model = LogisticRegression() # Instantiate the model
logistic_model.fit(X_train, y_train) # Fit the model

# Predictions
y_pred_logistic = logistic_model.predict(X_test)

# Evaluate logistic regression model
print("Logistic Regression Metrics:")
print("Accuracy:", accuracy_score(y_test, y_pred_logistic))
print("Precision:", precision_score(y_test, y_pred_logistic))
print("Recall:", recall_score(y_test, y_pred_logistic))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_logistic))

# Decision Tree model
decision_tree_model = DecisionTreeClassifier()
decision_tree_model.fit(X_train, y_train)

# Predictions
y_pred_tree = decision_tree_model.predict(X_test)

# Evaluate decision tree model
print("\nDecision Tree Metrics:")
print("Accuracy:", accuracy_score(y_test, y_pred_tree))
print("Precision:", precision_score(y_test, y_pred_tree))
print("Recall:", recall_score(y_test, y_pred_tree))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_tree))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

• (venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> & C:/Use

Logistic Regression Metrics:

Accuracy: 1.0

Precision: 1.0

Recall: 1.0

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	12
1.0	1.00	1.00	1.00	8
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

Decision Tree Metrics:

Accuracy: 1.0

Precision: 1.0

Recall: 1.0

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	12
1.0	1.00	1.00	1.00	8
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

○ (venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> █

Practical 8

Aim: Apply the K-Means algorithm to group similar data points into clusters, determine the optimal number of clusters using the elbow method or silhouette analysis, and visualize the clustering results.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load the Iris dataset
iris = load_iris()
iris_df = pd.DataFrame(data= np.c_[iris['data'], iris['target']], columns=
iris['feature_names'] + ['target'])

# Selecting features
X = iris_df.drop('target', axis=1)

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Determine the optimal number of clusters using the elbow method
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the elbow method
plt.figure(figsize=(8,6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS') # Within-cluster sum of squares
plt.show()

# Based on the elbow method, let's choose the optimal number of clusters as 3

# Apply K-Means clustering with the optimal number of clusters
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
kmeans.fit(X_scaled)

# Perform PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Get cluster centroids and labels
centroids = kmeans.cluster_centers_
labels = kmeans.labels_

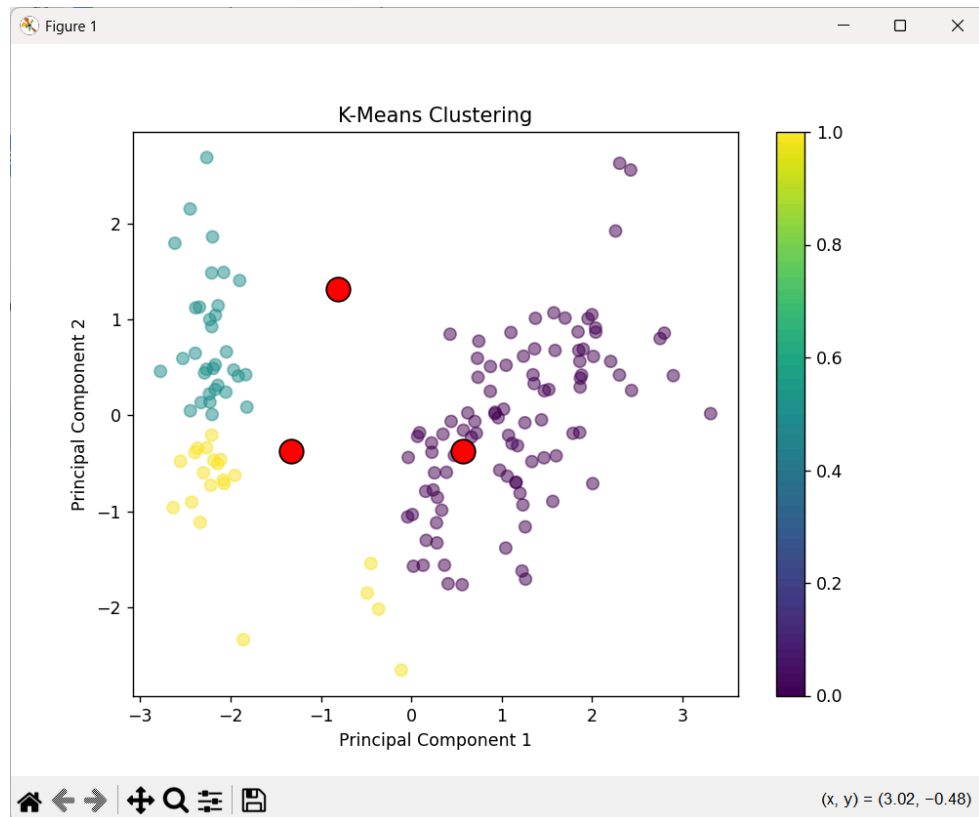
# Plot the clustering results
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', s=50,
alpha=0.5)
```

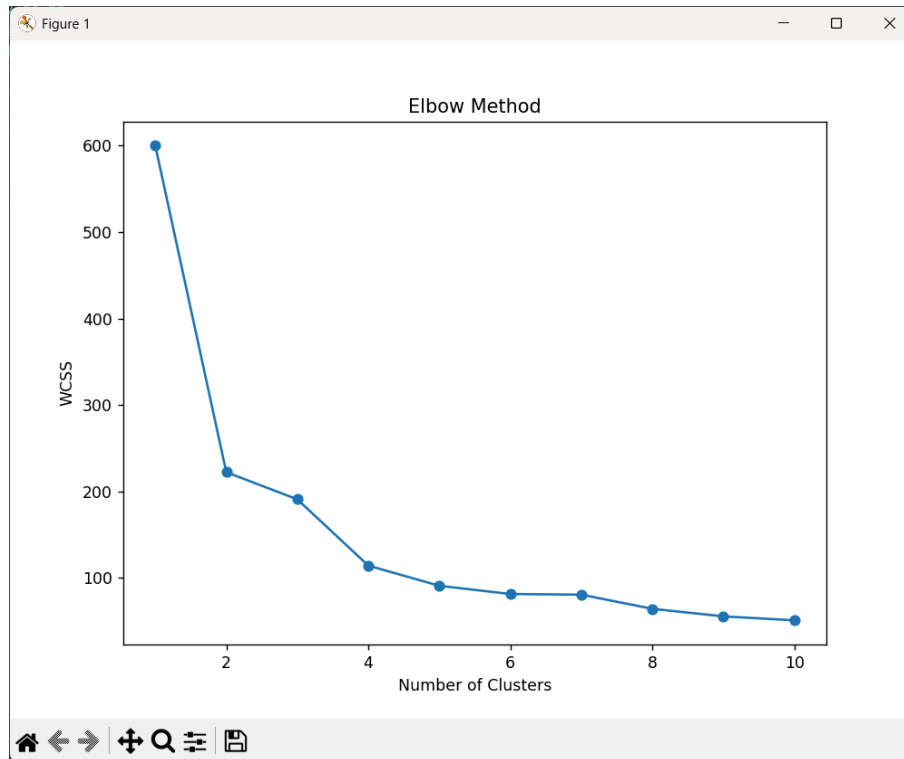


```
plt.scatter(centroids[:, 0], centroids[:, 1], marker='o', c='red', s=200,
edgecolor='k')
plt.title('K-Means Clustering')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar()
plt.show()

# Analyze cluster characteristics
cluster_df = iris_df.copy()
cluster_df['cluster'] = labels

# Displaying characteristics of each cluster
for cluster in sorted(cluster_df['cluster'].unique()):
    print(f"\nCluster {cluster}:")
    print(cluster_df[cluster_df['cluster'] == cluster].describe())
```





```
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> & C:/Users/tembu/OneDrive/Desktop/DataScience/venv
```

Cluster 0:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	cluster
count	96.000000	96.000000	96.000000	96.000000	96.000000	96.0
mean	6.314583	2.895833	4.973958	1.703125	1.520833	0.0
std	0.622724	0.315534	0.769586	0.411532	0.502188	0.0
min	4.900000	2.200000	3.500000	1.000000	1.000000	0.0
25%	5.800000	2.700000	4.400000	1.375000	1.000000	0.0
50%	6.300000	2.900000	4.900000	1.650000	2.000000	0.0
75%	6.700000	3.100000	5.600000	2.000000	2.000000	0.0
max	7.900000	3.800000	6.900000	2.500000	2.000000	0.0

Cluster 1:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	cluster
count	33.000000	33.000000	33.000000	33.000000	33.0	33.0
mean	5.169697	3.630303	1.493939	0.272727	0.0	1.0
std	0.288839	0.270976	0.181899	0.115306	0.0	0.0
min	4.600000	3.300000	1.000000	0.100000	0.0	1.0
25%	5.000000	3.400000	1.400000	0.200000	0.0	1.0
50%	5.100000	3.500000	1.500000	0.200000	0.0	1.0
75%	5.400000	3.800000	1.600000	0.400000	0.0	1.0
max	5.800000	4.400000	1.900000	0.600000	0.0	1.0

Cluster 2:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	cluster
count	21.000000	21.000000	21.000000	21.000000	21.000000	21.0
mean	4.747619	2.895238	1.757143	0.352381	0.190476	2.0
std	0.240040	0.361215	0.769137	0.338554	0.402374	0.0
min	4.300000	2.000000	1.100000	0.100000	0.000000	2.0
25%	4.600000	2.900000	1.300000	0.200000	0.000000	2.0
50%	4.800000	3.000000	1.400000	0.200000	0.000000	2.0
75%	4.900000	3.100000	1.600000	0.300000	0.000000	2.0
max	5.100000	3.200000	3.500000	1.100000	1.000000	2.0

```
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> 
```

Practical 9

Aim: Perform PCA on a dataset to reduce dimensionality, evaluate the explained variance, and visualize the data in the reduced-dimensional space.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load the Iris dataset
iris = load_iris()

# Create a DataFrame
iris_df = pd.DataFrame(data=np.c_[iris['data'], iris['target']],
                        columns=iris['feature_names'] + ['target'])

# Selecting features and target variable
X = iris_df.drop('target', axis=1)
y = iris_df['target']

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Perform PCA
pca = PCA()
Xpca = pca.fit_transform(X_scaled)

# Explained variance ratio
explained_variance_ratio = pca.explained_variance_ratio_

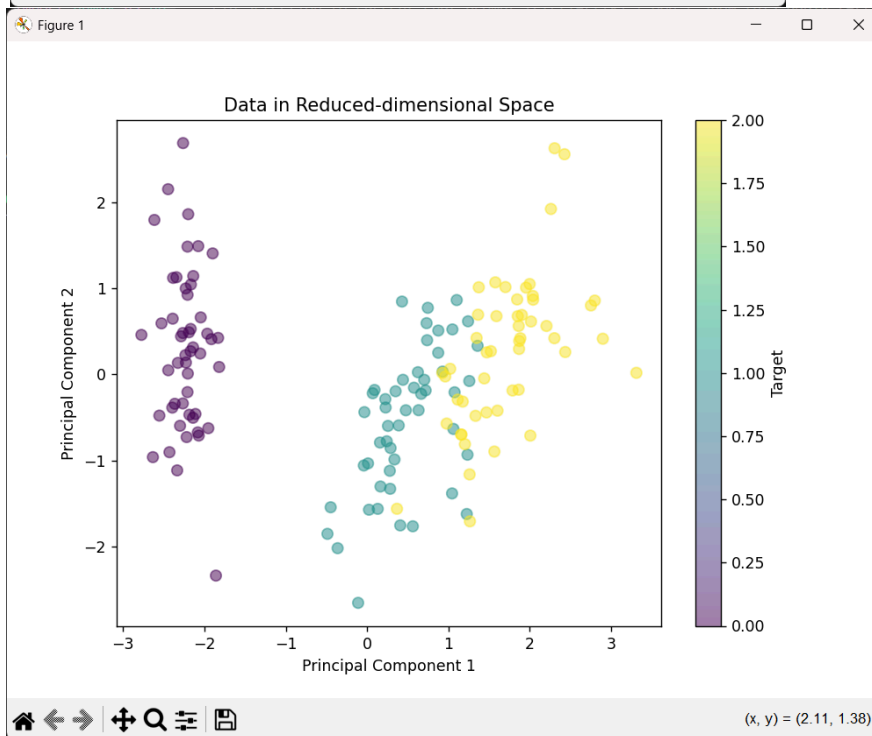
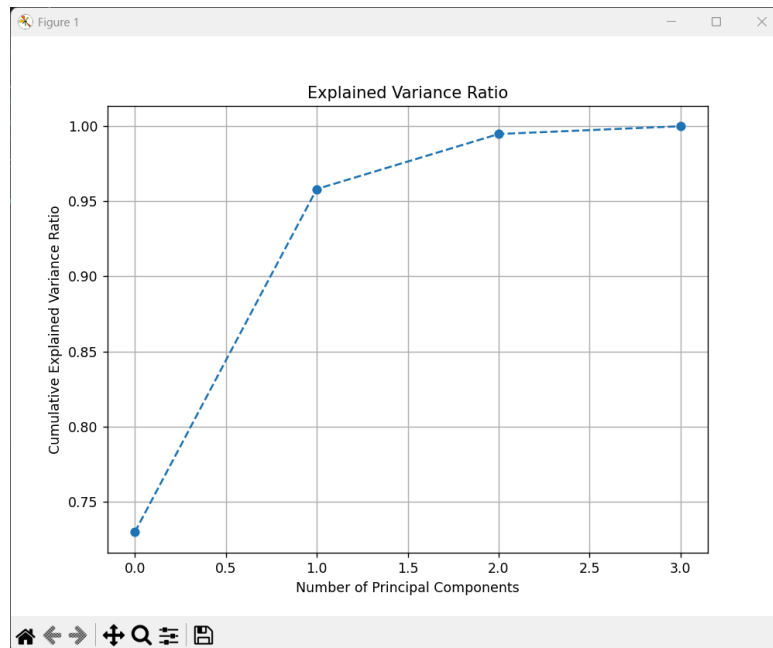
# Plot explained variance ratio
plt.figure(figsize=(8, 6))
plt.plot(np.cumsum(explained_variance_ratio), marker='o',
         linestyle='--')
plt.title('Explained Variance Ratio')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance Ratio')
plt.grid(True)
plt.show()

# Select the appropriate number of principal components
cumulative_variance_ratio = np.cumsum(explained_variance_ratio)
n_components = np.argmax(cumulative_variance_ratio >= 0.95) + 1

print(f"Number of principal components to explain 95% variance:
{n_components}")

# Reduce dimensionality using the selected number of principal
components
pca = PCA(n_components=n_components)
X_reduced = pca.fit_transform(X_scaled)
```

```
# Visualize data in the reduced-dimensional space
plt.figure(figsize=(8, 6))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y, cmap='viridis', s=50,
alpha=0.5)
plt.title('Data in Reduced-dimensional Space')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label="Target")
plt.show()
```



```
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> & C:/U
Number of principal components to explain 95% variance: 2
(venv) PS C:\Users\tembu\OneDrive\Desktop\DataScience> 
```

Practical 10

Aim: Create meaningful visualizations using data visualization tools, combine multiple visualizations to tell a compelling data story, and present findings and insights in a clear and concise manner.

 P10.csv

```
1  gender,churn,age,service_type,tenure,feature1,feature2,cluster
2  Male,1,35,Internet,12,5.1,2.2,1
3  Female,0,28,Phone,24,4.8,2.5,2
4  Male,1,45,Internet,6,6.3,3.1,1
5  Female,0,32,Internet,15,5.4,2.8,3
6  Male,0,60,Phone,36,5.9,2.9,2
7  Female,1,22,Internet,3,5.2,2.3,1
8  Male,0,40,Phone,20,4.7,2.4,3
9  Female,1,55,Internet,10,6.0,2.7,1
10 Male,0,50,Phone,30,5.5,2.6,2
11 Female,1,30,Internet,18,5.3,2.9,1
12
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import pandas as pd

# Load dataset
df = pd.read_csv('P10.csv') # Change the path to your dataset

# Check column names to ensure the expected columns exist
print(df.columns)

# Ensure 'churn' is numeric and handle any missing values
df['churn'] = pd.to_numeric(df['churn'], errors='coerce')

# Churn rate by gender (using seaborn)
if 'gender' in df.columns and 'churn' in df.columns:
    sns.countplot(data=df, x='gender', hue='churn')
    plt.title('Churn Rate by Gender')
    plt.show()

# Churn rate by age group (using plotly)
if 'age' in df.columns and 'churn' in df.columns:
    fig = px.histogram(df, x='age', color='churn', nbins=20,
histnorm='percent')
    fig.update_layout(title='Churn Rate by Age Group',
xaxis_title='Age', yaxis_title='% of Customers')
    fig.write_html("churn_by_age.html", auto_open=True)

# Churn rate by service type (using matplotlib)
if 'service_type' in df.columns and 'churn' in df.columns:
```

```

    service_churn = df.groupby('service_type')['churn'].mean()
    plt.pie(service_churn, labels=service_churn.index,
    autopct='%1.1f%%')
    plt.title('Churn Rate by Service Type')
    plt.show()

# Correlation matrix (using seaborn) - Now we filter only numeric
columns
numeric_columns = df.select_dtypes(include=['number']).columns # Get
only numeric columns
if len(numeric_columns) > 1: # Ensure there's more than one numeric
column
    correlation_matrix = df[numeric_columns].corr()
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
    plt.title('Correlation Matrix')
    plt.show()

# Customer tenure vs. churn (using plotly)
if 'tenure' in df.columns and 'churn' in df.columns:
    fig = px.scatter(df, x='tenure', y='churn', color='churn')
    fig.update_layout(title='Customer Tenure vs. Churn')
    xaxis_title="Tenure", yaxis_title='Churn (1=Yes, 0=No)'
    fig.write_html("tenure_vs_churn.html", auto_open=True)

# Customer segmentation (using plotly)
if 'feature1' in df.columns and 'feature2' in df.columns and 'cluster'
in df.columns:
    fig = px.scatter(df, x='feature1', y='feature2', color='cluster')
    fig.update_layout(title='Customer Segmentation')
    fig.write_html("segmentation_plot.html", auto_open=True)

```

